🚀 منصة مساري - نظام تشغيل الحياة الشخصية
برومبت البناء الشامل

🎯 الهوية والسياق

اسم المشروع: مساري
الوصف: نظام تشغيل حياة شخصية متكامل يربط كل جوانب الحياة في منصة واحدة
المستخدم المستهدف: طالب توجيهي صناعي (تخصص كهرباء استعمال) من الأردن
الهدف الرئيسي: السفر للصين للدراسة الجامعية بعد التوجيهي
اللغة: عربي بالكامل مع دعم RTL إجباري
الفلسفة: AI Agent هو العقل الرئيسي للمنصة ويتحكم في كل شيء

📦 التقنيات والأدوات

Frontend: Next.js 16 مع App Router و TypeScript، Server Components في وضع Strict، Tailwind Framer Motion، shadcn/ui components، CSS 4 للحركات، Recharts للرسوم البيانية، Zustand لإدارة الحالة

Backend: Next.js API Routes و SQLite، Prisma ORM، Server Actions للتطوير ثم PostgreSQL للإنتاج، z-ai-web-dev-sdk للذكاء الاصطناعي مع streaming و tool calling

Fonts: Cairo للعربية، JetBrains Mono للكود

Design System: Linear-style dark theme، clean و RTL support، minimal

---

🗄️ قاعدة البيانات الكاملة

أنشئ ملف prisma/schema.prisma بالمحتوى التالي:

```
{ "generator client { provider = "prisma-client-js
{ "datasource db { provider = "sqlite" url = "file:./dev.db

} model User
(()id String @id @default(cuid
```

```
  name String @default("عبد العزيز")
  email String? @unique
  avatar String?
  tawjihiDate DateTime?
  chinaDate DateTime?
  level Int @default(1)
  xp Int @default(0)
  xpToNextLevel Int @default(100)
  class String @default("warrior")
  title String @default("مبتدئ")
  gold Int @default(0)
  gems Int @default(0)
  totalTasksCompleted Int @default(0)
  totalHabitsLogged Int @default(0)
  currentStreak Int @default(0)
  longestStreak Int @default(0)
  lastActiveDate DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  habits Habit[]
  tasks Task[]
  goals Goal[]
  journals Journal[]
  moodLogs MoodLog[]
  transactions Transaction[]
  achievements UserAchievement[]
  conversations AIConversation[]
  settings Setting?
}

model Habit {
  id String @id @default(cuid())
  userId String
  name String
  description String?
  icon String @default("⭕")
  color String @default("#3b82f6")
  frequency String @default("daily")
  streak Int @default(0)
  bestStreak Int @default(0)
  totalCompletions Int @default(0)
  freezes Int @default(3)
  xpPerCompletion Int @default(10)
  isActive Boolean @default(true)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
  completions HabitCompletion[]
```

```
([index([userId@@
{

} model HabitCompletion
(()id String @id @default(cuid
habitId String
date DateTime @db.Date
(completed Boolean @default(false
?note String
(xpEarned Int @default(0
(()createdAt DateTime @default(now
(habit Habit @relation(fields: [habitId], references: [id], onDelete: Cascade
([unique([habitId, date@@
([index([habitId, date@@
{

} model Task
(()id String @id @default(cuid
userId String
title String
?description String
("status String @default("pending
("priority String @default("medium
?category String
?dueDate DateTime
?completedAt DateTime
(xpReward Int @default(10
(xpEarned Int @default(0
(()createdAt DateTime @default(now
updatedAt DateTime @updatedAt
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
([index([userId, status@@
([index([userId, dueDate@@
{

} model Goal
(()id String @id @default(cuid
userId String
title String
?description String
("category String @default("personal
(progress Int @default(0
("status String @default("active
?targetDate DateTime
(xpReward Int @default(100
(()createdAt DateTime @default(now
updatedAt DateTime @updatedAt
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
```

```
[]milestones Milestone
([index([userId, status@@
{

} model Milestone
(()id String @id @default(cuid
goalId String
title String
(completed Boolean @default(false
?completedAt DateTime
(order Int @default(0
(goal Goal @relation(fields: [goalId], references: [id], onDelete: Cascade
([index([goalId@@
{

} model MoodLog
(()id String @id @default(cuid
userId String
mood Int
?energy Int
?stress Int
?sleep Int
?note String
date DateTime @db.Date
(()createdAt DateTime @default(now
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
([unique([userId, date@@
([index([userId, date@@
{

} model Journal
(()id String @id @default(cuid
userId String
?title String
content String
?tags String
date DateTime @db.Date
(()createdAt DateTime @default(now
updatedAt DateTime @updatedAt
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
([index([userId, date@@
{

} model Transaction
(()id String @id @default(cuid
userId String
type String
amount Float
```

```
category String
?description String
date DateTime @db.Date
(()createdAt DateTime @default(now
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
([index([userId, date@@
([index([userId, type@@
{

} model Achievement
(()id String @id @default(cuid
key String @unique
name String
description String
icon String
category String
xpReward Int
condition String
(isSecret Boolean @default(false
(()createdAt DateTime @default(now
[]users UserAchievement
{

} model UserAchievement
userId String
achievementId String
(()unlockedAt DateTime @default(now
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
([achievement Achievement @relation(fields: [achievementId], references: [id
([id([userId, achievementId@@
{

} model AIConversation
(()id String @id @default(cuid
userId String
?title String
messages String
?context String
(()createdAt DateTime @default(now
updatedAt DateTime @updatedAt
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
([index([userId@@
{

} model Setting
(()id String @id @default(cuid
userId String @unique
("theme String @default("dark
```

```
("accentColor String @default("blue
(notifications Boolean @default(true
("language String @default("ar
?aiApiKey String
(()createdAt DateTime @default(now
updatedAt DateTime @updatedAt
(user User @relation(fields: [userId], references: [id], onDelete: Cascade
{
```

بعد إنشاء Schema شغل: npx prisma migrate dev --name init ثم npx prisma db seed

---

🎨 نظام التصميم

أنشئ app/globals.css:

```
import@
url('https://fonts.googleapis.com/css2?family=Cairo:wght@300;400;500;600;700;800&family
;('=JetBrains+Mono:wght@400;500;600&display=swap

} layer base@
} root:
;background: 9 9 11--
;foreground: 250 250 250--
;card: 24 24 27--
;card-foreground: 250 250 250--
;primary: 59 130 246--
;primary-foreground: 255 255 255--
;secondary: 38 38 41--
;secondary-foreground: 250 250 250--
;muted: 39 39 42--
;muted-foreground: 161 161 170--
;accent: 59 130 246--
;accent-foreground: 255 255 255--
;destructive: 239 68 68--
;destructive-foreground: 255 255 255--
;border: 39 39 42--
;input: 39 39 42--
;ring: 59 130 246--
;radius: 0.5rem--

;success: 34 197 94--
;warning: 245 158 11--
;purple: 168 85 247--
;cyan: 6 182 212--
```

```
{

{ ;((border-color: hsl(var(--border } *

} body
;((background: hsl(var(--background
;((color: hsl(var(--foreground
;font-family: 'Cairo', sans-serif
;direction: rtl
;text-align: right
{

{ ;code, pre { font-family: 'JetBrains Mono', monospace
{

} layer components@
} stat-card.
;((background: hsl(var(--card
;((border: 1px solid hsl(var(--border
;(border-radius: var(--radius
;padding: 1.5rem
;transition: all 0.2s
{

} stat-card:hover.
;(border-color: hsl(var(--primary) / 0.5
;(box-shadow: 0 0 20px hsl(var(--primary) / 0.1
{

} glow-text.
;text-shadow: 0 0 10px currentColor, 0 0 20px currentColor
{

} progress-bar.
;((background: hsl(var(--muted
;border-radius: 9999px
;overflow: hidden
{

} progress-fill.
;(((background: linear-gradient(90deg, hsl(var(--primary)), hsl(var(--cyan
;height: 100%
;border-radius: 9999px
;transition: width 0.5s ease
{
{
```

📁 هيكل الملفات المطلوب

```
/app
├── layout.tsx ← الـ layout الرئيسي مع RTL والخطوط والـ providers
├── Dashboard ← page.tsx
├── AI Agent ← agent/page.tsx
├── habits/page.tsx ← تتبع العادات
├── tasks/page.tsx ← إدارة المهام
├── finance/page.tsx ← المالية
├── mental/page.tsx ← الصحة النفسية
├── rpg/page.tsx ← نظام RPG
├── goals/page.tsx ← الأهداف
├── china/page.tsx ← خطة الصين
├── stats/page.tsx ← الإحصائيات
├── settings/page.tsx ← الإعدادات
├── /api
│   ├── user/route.ts ← GET/PUT للمستخدم
│   ├── habits/route.ts ← GET/POST/PUT/DELETE للعادات
│   ├── habits/[id]/complete/route.ts ← POST لتسجيل إكمال عادة
│   ├── tasks/route.ts ← CRUD للمهام
│   ├── tasks/[id]/complete/route.ts ← POST لإكمال مهمة
│   ├── transactions/route.ts ← CRUD للمعاملات
│   ├── mood/route.ts ← GET/POST للمزاج
│   ├── goals/route.ts ← CRUD للأهداف
│   ├── goals/[id]/progress/route.ts ← PUT لتحديث التقدم
│   ├── agent/route.ts ← POST للـ AI Agent مع streaming
│   └── stats/route.ts ← GET للإحصائيات

/lib
├── prisma.ts ← Prisma client singleton
├── ai-tools.ts ← أدوات الـ AI Agent
├── xp-system.ts ← حساب XP والترقية
├── streak-system.ts ← حساب الـ streaks

/components
├── /layout
│   ├── sidebar.tsx ← القائمة الجانبية
│   ├── header.tsx ← الهيدر
│   └── container ← page-container.tsx للصفحات
├── shadcn components ← /ui
├── /dashboard
│   ├── countdown-card.tsx ← بطاقة العداد التنازلي
│   ├── stat-card.tsx ← بطاقة الإحصائية
│   └── quick-actions.tsx ← الأزرار السريعة
```

```
├── /habits
│   ├── habit-list.tsx ← قائمة العادات
│   ├── habit-card.tsx ← بطاقة عادة
│   └── habit-heatmap.tsx ← الخريطة الحرارية
├── /tasks
│   ├── task-list.tsx ← قائمة المهام
│   └── task-card.tsx ← بطاقة مهمة
├── /finance
│   ├── balance-card.tsx ← بطاقة الرصيد
│   ├── transaction-form.tsx ← نموذج إضافة معاملة
│   └── spending-chart.tsx ← رسم الإنفاق
├── /mental
│   ├── mood-selector.tsx ← اختيار المزاج
│   ├── mood-calendar.tsx ← تقويم المزاج
│   └── journal-editor.tsx ← محرر المذكرات
├── /rpg
│   ├── character-card.tsx ← بطاقة الشخصية
│   ├── xp-bar.tsx ← شريط الخبرة
│   └── achievements-grid.tsx ← شبكة الإنجازات
├── /goals
│   ├── goal-list.tsx ← قائمة الأهداف
│   ├── goal-card.tsx ← بطاقة هدف
│   └── milestone-list.tsx ← قائمة المراحل
├── /agent
│   ├── chat-interface.tsx ← واجهة المحادثة
│   ├── message-bubble.tsx ← فقاعة الرسالة
│   └── quick-prompts.tsx ← الرسائل السريعة
└── /shared
    ├── countdown-timer.tsx ← العداد التنازلي
    ├── progress-ring.tsx ← حلقة التقدم
    ├── empty-state.tsx ← حالة الفراغ
    └── loading-spinner.tsx ← مؤشر التحميل

/stores
├── user-store.ts ← حالة المستخدم
└── ui-store.ts ← حالة الواجهة
```

🤖 AI Agent - العقل الرئيسي

أنشئ lib/ai-tools.ts:

```
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()
```

```
} = export const aiTools
// ========== إدارة المهام ==========
} :create_task
:description 'إنشاء مهمة جديدة للمستخدم',
} :parameters
'type: 'object,
} :properties
:title { type: 'string', description 'عنوان المهمة' },
:description: { type: 'string', description 'وصف المهمة' },
:priority: { type: 'string', enum: ['low', 'medium', 'high', 'urgent'], description 'الأولوية' },
:due_date: { type: 'string', description 'تاريخ الاستحقاق بصيغة ISO' },
:category: { type: 'string', description 'فئة المهمة' }
},
['required: ['title
},
} <= (execute: async (params: any, userId: string
const xpReward = params.priority === 'urgent' ? 25 : params.priority === 'high' ? 20 :
params.priority === 'medium' ? 15 : 10
})const task = await prisma.task.create
} :data
,userId
,title: params.title
,description: params.description
,'priority: params.priority || 'medium
,category: params.category
,dueDate: params.due_date ? new Date(params.due_date) : null
xpReward
{
({
:return { success: true, task, message `تم إنشاء المهمة "${params.title}"! مكافأة: ${xpReward}`
{ `XP
{
},

} :complete_task
:description 'إكمال مهمة ومنح XP للمستخدم',
} :parameters
'type: 'object,
:properties: { task_id: { type: 'string', description 'معرف المهمة' } },
['required: ['task_id
},
} <= (execute: async (params: any, userId: string
({ { const task = await prisma.task.findFirst({ where: { id: params.task_id, userId
:if (!task) return { success: false, message 'المهمة غير موجودة' }
:if (task.status === 'completed') return { success: false, message 'المهمة مكتملة بالفعل' }

})await prisma.task.update
,{ where: { id: params.task_id
```

```
({
{ data: { status: 'completed', completedAt: new Date(), xpEarned: task.xpReward

({ const user = await prisma.user.findUnique({ where: { id: userId
} (if (user
const newXp = user.xp + task.xpReward
const leveledUp = newXp >= user.xpToNextLevel
})await prisma.user.update
,{ where: { id: userId
} :data
,xp: leveledUp ? newXp - user.xpToNextLevel : newXp
,level: leveledUp ? user.level + 1 : user.level
xpToNextLevel: leveledUp ? Math.floor(user.xpToNextLevel * 1.5) :
,user.xpToNextLevel
totalTasksCompleted: user.totalTasksCompleted + 1
{
({
if (leveledUp) return { success: true, xpEarned: task.xpReward, leveledUp: true,
:newLevel: user.level + 1, message: `🎉 مبروك! ارتفعت للمستوى ${user.level + 1}! `
{
:return { success: true, xpEarned: task.xpReward, message: `تم إكمال المهمة! حصلت على
{ `task.xpReward} XP}$
{
,{

} :list_tasks
:description: 'عرض قائمة المهام',
} :parameters
,'type: 'object
} :properties
,{ ['status: { type: 'string', enum: ['pending', 'completed', 'all
{ 'عدد المهام' :limit: { type: 'number', description
{
,{
} <= (execute: async (params: any, userId: string
})const tasks = await prisma.task.findMany
('where: { userId, status: params.status === 'all' ? undefined : (params.status || 'pending
,{
,[{ 'orderBy: [{ priority: 'desc' }, { dueDate: 'asc
take: params.limit || 10
({
{ return { success: true, count: tasks.length, tasks
{
,{

// ========== إدارة العادات ==========
} :create_habit
:description: 'إنشاء عادة جديدة',
```

```
} :parameters
,'type: 'object
} :properties
name: { type: 'string', description: 'اسم العادة' },
description: { type: 'string', description: 'وصف العادة' },
icon: { type: 'string', description: 'إيموجي للعادة' },
color: { type: 'string', description: 'لون العادة بصيغة hex' }
},
['required: ['name
},
} <= (execute: async (params: any, userId: string
})const habit = await prisma.habit.create
} :data
,userId
,name: params.name
,description: params.description
,'⭕' || icon: params.icon
'color: params.color || '#3b82f6
{
({
return { success: true, habit, message: `تم إنشاء العادة "${params.name}"!` }
{
},

} :log_habit
description: 'تسجيل إكمال عادة لليوم',
} :parameters
,'type: 'object
} :properties
habit_id: { type: 'string', description: 'معرف العادة' },
note: { type: 'string', description: 'ملاحظة' }
},
['required: ['habit_id
},
} <= (execute: async (params: any, userId: string
(const today = new Date(); today.setHours(0, 0, 0, 0
({ { const habit = await prisma.habit.findFirst({ where: { id: params.habit_id, userId
if (!habit) return { success: false, message: 'العادة غير موجودة' }

})const existing = await prisma.habitCompletion.findUnique
{ { where: { habitId_date: { habitId: params.habit_id, date: today
({
if (existing?.completed) return { success: false, message: 'تم تسجيل هذه العادة اليوم بالفعل' }

})await prisma.habitCompletion.upsert
,{ { where: { habitId_date: { habitId: params.habit_id, date: today
create: { habitId: params.habit_id, date: today, completed: true, note: params.note,
,{ xpEarned: habit.xpPerCompletion
```

```
{ update: { completed: true, note: params.note, xpEarned: habit.xpPerCompletion
({

(const yesterday = new Date(today); yesterday.setDate(yesterday.getDate() - 1
})const yesterdayCompletion = await prisma.habitCompletion.findUnique
{ { where: { habitId_date: { habitId: params.habit_id, date: yesterday
({
const newStreak = yesterdayCompletion?.completed ? habit.streak + 1 : 1

})await prisma.habit.update
,{ where: { id: params.habit_id
data: { streak: newStreak, bestStreak: Math.max(newStreak, habit.bestStreak),
{ totalCompletions: habit.totalCompletions + 1
({

({ { const user = await prisma.user.findUnique({ where: { id: userId
} (if (user
})await prisma.user.update
,{ where: { id: userId
data: { xp: user.xp + habit.xpPerCompletion, totalHabitsLogged:
{ user.totalHabitsLogged + 1
({
{

return { success: true, streak: newStreak, xpEarned: habit.xpPerCompletion, message:
{ `يوم {newStreak}$ :السلسلة تم! `: `يوم! {newStreak}$ :سلسلة جديدة 🔥 ` ? newStreak > habit.streak
{
,{

} :list_habits
description: 'عرض قائمة العادات مع حالة اليوم',
,{ {} :parameters: { type: 'object', properties
} <= (execute: async (params: any, userId: string
(const today = new Date(); today.setHours(0, 0, 0, 0
})const habits = await prisma.habit.findMany
,{ where: { userId, isActive: true
{ { { include: { completions: { where: { date: today
({
} return
,success: true
,count: habits.length
}) <= habits: habits.map(h
,id: h.id, name: h.name, icon: h.icon, streak: h.streak, bestStreak: h.bestStreak
completedToday: h.completions[0]?.completed || false
(({
{
{
,{
```

```
// ========== المالية ==========
add_transaction: {
description: 'إضافة معاملة مالية',
parameters: {
type: 'object',
properties: {
type: { type: 'string', enum: ['income', 'expense'], description: 'نوع المعاملة' },
amount: { type: 'number', description: 'المبلغ' },
category: { type: 'string', description: 'الفئة (food, transport, savings, etc.)' },
description: { type: 'string', description: 'وصف' }
},
required: ['type', 'amount', 'category']
},
execute: async (params: any, userId: string) => {
const transaction = await prisma.transaction.create({
data: { userId, type: params.type, amount: params.amount, category: params.category,
description: params.description, date: new Date() }
})
return { success: true, transaction, message: `تم تسجيل ${params.type === 'income' ? 'الدخل' : 'المصروف'}: ${params.amount}$` }
}
},

get_balance: {
description: 'عرض الرصيد والإحصائيات المالية',
parameters: { type: 'object', properties: {} },
execute: async (params: any, userId: string) => {
const transactions = await prisma.transaction.findMany({ where: { userId } })
const income = transactions.filter(t => t.type === 'income').reduce((sum, t) => sum + t.amount, 0)
const expenses = transactions.filter(t => t.type === 'expense').reduce((sum, t) => sum + t.amount, 0)
const savings = transactions.filter(t => t.category === 'savings').reduce((sum, t) => sum + t.amount, 0)
return { success: true, balance: income - expenses, totalIncome: income, totalExpenses: expenses, savings }
}
},

// ========== الصحة النفسية ==========
log_mood: {
description: 'تسجيل المزاج اليومي',
parameters: {
type: 'object',
properties: {
mood: { type: 'number', description: 'المزاج من 1 (سئ) إلى 5 (ممتاز)' },
energy: { type: 'number', description: 'الطاقة من 1 إلى 5' },
```

```
stress: { type: 'number', description: 'التوتر من 1 إلى 5' },
note: { type: 'string', description: 'ملاحظة' }
},
required: ['mood']
},
execute: async (params: any, userId: string) => {
const today = new Date(); today.setHours(0, 0, 0, 0);
const moodLog = await prisma.moodLog.upsert({
where: { userId_date: { userId, date: today } },
create: { userId, date: today, mood: params.mood, energy: params.energy, stress: params.stress, note: params.note },
update: { mood: params.mood, energy: params.energy, stress: params.stress, note: params.note }
});
const emoji = ['😢', '😕', '😐', '🙂', '😊'][params.mood - 1];
return { success: true, moodLog, message: `تم تسجيل مزاجك: ${emoji}` }
}
},

// ========== الأهداف ==========
create_goal: {
description: 'إنشاء هدف جديد',
parameters: {
type: 'object',
properties: {
title: { type: 'string', description: 'عنوان الهدف' },
description: { type: 'string', description: 'وصف الهدف' },
category: { type: 'string', enum: ['personal', 'career', 'health', 'financial', 'learning'], description: 'فئة الهدف' },
target_date: { type: 'string', description: 'التاريخ المستهدف' }
},
required: ['title']
},
execute: async (params: any, userId: string) => {
const goal = await prisma.goal.create({
data: { userId, title: params.title, description: params.description, category: params.category || 'personal', targetDate: params.target_date ? new Date(params.target_date) : null }
});
return { success: true, goal, message: `تم إنشاء الهدف "${params.title}"!` }
}
},

update_goal_progress: {
description: 'تحديث تقدم الهدف',
parameters: {
type: 'object',
```

```
progress: { type: 'number', ,{ 'معرف الهدف' :properties: { goal_id: { type: 'string', description
,{ { '100-0 النسبة' :description
['required: ['goal_id', 'progress
,{
} <= (execute: async (params: any, userId: string
({ { const goal = await prisma.goal.findFirst({ where: { id: params.goal_id, userId
{ 'الهدف غير موجود' :if (!goal) return { success: false, message

((const progress = Math.min(100, Math.max(0, params.progress
})await prisma.goal.update
,{ where: { id: params.goal_id
data: { progress, status: progress >= 100 ? 'completed' : 'active', completedAt: progress
{ >= 100 ? new Date() : null
({

} ('if (progress >= 100 && goal.status !== 'completed
({ { const user = await prisma.user.findUnique({ where: { id: userId
if (user) await prisma.user.update({ where: { id: userId }, data: { xp: user.xp +
({ { goal.xpReward
`🎉 مبروك! أتممت الهدف :return { success: true, xpEarned: goal.xpReward, message
{ `goal.title}"! +${goal.xpReward} XP}$"
{
{ `%{progress}$ تم تحديث التقدم إلى :return { success: true, message
{
,{

// ========== المستخدم ==========
} :get_user_stats
'عرض إحصائيات المستخدم الكاملة' :description,
,{ {} :parameters: { type: 'object', properties
} <= (execute: async (params: any, userId: string
})const user = await prisma.user.findUnique
,{ where: { id: userId
include: { habits: { where: { isActive: true } }, tasks: { where: { status: 'pending' } },
{ achievements: true
({
{ 'المستخدم غير موجود' :if (!user) return { success: false, message
} return
,success: true
} :stats
level: user.level, xp: user.xp, xpToNextLevel: user.xpToNextLevel, title: user.title, class:
,user.class
gold: user.gold, gems: user.gems, currentStreak: user.currentStreak, longestStreak:
,user.longestStreak
totalTasksCompleted: user.totalTasksCompleted, totalHabitsLogged:
,user.totalHabitsLogged
activeHabits: user.habits.length, pendingTasks: user.tasks.length, achievements:
user.achievements.length
```

{
{
{
{
{

أنشئ app/api/agent/route.ts للـ streaming مع tool calling:

```
import ZAI from 'z-ai-web-dev-sdk'
import { aiTools } from '@/lib/ai-tools'
import { prisma } from '@/lib/prisma'
import { NextRequest } from 'next/server'

export async function POST(req: NextRequest) {
  const { messages } = await req.json()
  const userId = 'default-user' // في التطبيق الحقيقي، استخدم session

  const zai = await ZAI.create()

  const systemPrompt = `أنت مساعد ذكي لمنصة "مساري" - نظام تتبع الحياة الشخصية.
يمكنك تنفيذ الأوامر التالية للمستخدم:
- create_task: إنشاء مهمة جديدة
- complete_task: إكمال مهمة
- list_tasks: عرض المهام
- create_habit: إنشاء عادة
- log_habit: تسجيل عادة
- list_habits: عرض العادات
- add_transaction: إضافة معاملة مالية
- get_balance: عرض الرصيد
- log_mood: تسجيل المزاج
- create_goal: إنشاء هدف
- update_goal_progress: تحديث تقدم هدف
- get_user_stats: عرض إحصائيات المستخدم

عندما يطلب المستخدم شيئاً، استخدم الأداة المناسبة.
أجب بالعربية دائماً. كن مختصراً ومفيداً.`

  const stream = new ReadableStream({
    start(controller) {
      const encoder = new TextEncoder()

      const response = await zai.chat.completions.create({
        messages: [{ role: 'system', content: systemPrompt }, ...messages],
        stream: true
      })

      for (const chunk of response) {
        const content = chunk.choices[0]?.delta?.content || ''
```

```
} (if (content
((`controller.enqueue(encoder.encode(`data: ${JSON.stringify({ content })}\n\n
{
{

(('controller.enqueue(encoder.encode('data: [DONE]\n\n
()controller.close
{
({

} ,return new Response(stream
headers: { 'Content-Type': 'text/event-stream', 'Cache-Control': 'no-cache', 'Connection':
{ "keep-alive
({
{
```

📱 تفاصيل الصفحات

1. Dashboard (/) - الصفحة الرئيسية

Header: مساء الخير، عبد العزيز | التاريخ | الوقت

العدادات التنازلية (2 بطاقات كبيرتين):
- التوجيهي: 93 يوم متبقي
- السفر للصين: 171 يوم متبقي

4) Quick Stats بطاقات):
- 🔥 الالتزام: 0 يوم
- 💰 التوفير: 0$
- 📊 تقدم ATS: 0%
- 🇨🇳 كلمات صينية: 0

ملخص اليوم:
- المهام المعلقة: 0
- العادات المتبقية: 0/0
- ساعات التركيز: 0

4) Quick Actions أزرار):
- 📝 سجل اليوم
- ✅ أضف مهمة
- 🤖 تحدث مع AI
- 🎯 سجل عادة

2. Agent (/agent) - الوكيل الذكي

:Chat Interface
- Messages list مع user و assistant bubbles
- Input area مع text input و send button
- Quick prompts: "أضف مهمة"، "أظهر مهامي"، "سجل عادة"، "إحصائياتي"

:Sidebar
- Conversation history
- New chat button

3. Habits (/habits) - العادات

:Header
- العنوان: "العادات"
- زر: "+ عادة جديدة"

قائمة العادات:
- كل عادة: icon + name + streak badge + check button
- عند الضغط على check: تسجيل الإكمال + تحديث streak

:Heatmap
- سنوي (365 يوم)
- الألوان: أخضر فاتح للإكمال، رمادي للغياب

4. Tasks (/tasks) - المهام

:Header
- العنوان: "المهام"
- فلترة: status (pending/completed/all), priority
- بحث
- زر: "+ مهمة جديدة"

قائمة المهام:
- checkbox + title + priority indicator + due date + XP reward
- عند الضغط على checkbox: إكمال المهمة + إضافة XP

نموذج إضافة مهمة:
- العنوان (مطلوب)
- الوصف
- الأولوية
- تاريخ الاستحقاق
- الفئة

5. Finance (/finance) - المالية

بطاقات Overview:
- الرصيد: $0
- الدخل الشهري: $0

- المصروفات الشهري: 0$
- نسبة الادخار: 0%

نموذج إضافة معاملة:
- النوع: دخل / مصروف
- المبلغ
- الفئة
- الوصف

Charts:
- Pie chart: الإنفاق حسب الفئة
- Line chart: الدخل vs المصروفات

هدف التوفير:
- Progress bar: $0 / $7,000

6. Mental (/mental) - الصحة النفسية

Mood Logger:
- اختيار المزاج: 5 emojis (😢 😕 😐 🙂 😊)
- مستوى الطاقة: 1-5 slider
- مستوى التوتر: 1-5 slider
- ملاحظة: textarea
- زر: "حفظ"

تقويم المزاج:
- Month view
- كل يوم ملون حسب المزاج

المذكرات:
- Today's entry
- Past entries list

7. RPG (/rpg) - نظام اللعب

بطاقة الشخصية:
- Avatar circle مع الحرف الأول
- الاسم: عبد العزيز
- المستوى: LV.1
- العنوان: مبتدئ
- Class: محارب

XP Bar:
- شريط تقدم
- النص: "XP: 0 / 100"

Stats:
- المهام المكتملة: 0

- العادات المسجلة: 0
- أطول سلسلة: 0

Achievements:
- Grid من البطاقات
- Unlocked: ملونة
- Locked: رمادية مع قفل

8. Goals (/goals) - الأهداف

Header:
- العنوان: "الأهداف"
- فلترة: category
- زر: "+ هدف جديد"

قائمة الأهداف:
- Title
- Progress bar
- Target date
- XP reward

نموذج إضافة هدف:
- العنوان
- الوصف
- الفئة
- التاريخ المستهدف

9. China (/china) - خطة الصين

العداد التنازلي:
- عدد الأيام المتبقية للسفر

Checklist التحضيرات:
- ✈️ التأشيرة
- 📄 الأوراق المطلوبة
- 💰 الادخار
- 🎓 التقديم الجامعي
- 🏠 السكن

تقدم التعلم:
- مستوى HSK
- الكلمات المتعلمة
- ساعات الدراسة

10. Stats (/stats) - الإحصائيات

Time Range Selector:
- اليوم

- هذا الأسبوع
- هذا الشهر
- مخصص

:Charts
- Activity heatmap
- Weekly productivity trend
- Tasks completion rate
- Habits streak history
- Mood trend

:Insights
- AI-generated insights
- Recommendations

11. Settings (/settings) - الإعدادات

:Profile
- الاسم
- الصورة الرمزية

:Appearance
- Theme: dark/light
- Accent color

:Notifications
- Enable/disable
- Quiet hours

:AI
- External API key (optional)

⚡ نظام RPG الكامل

XP Rewards:
- إكمال عادة يومية: 10 XP
- إكمال مهمة low priority: 10 XP
- إكمال مهمة medium priority: 15 XP
- إكمال مهمة high priority: 20 XP
- إكمال مهمة urgent priority: 25 XP
- تسجيل المزاج: 5 XP
- إكمال هدف: 100 XP

Level Up:

- المستوى 1: 100 XP للحصول على المستوى 2
- المستوى 2: 150 XP للحصول على المستوى 3
- كل مستوى: XP المطلوب = 100 × المستوى × 1.5

Titles:
- المستوى 1-5: مبتدئ
- المستوى 6-10: متدرب
- المستوى 11-20: محارب
- المستوى 21-30: بطول
- المستوى 31-50: أسطوري
- المستوى 51+: خرافي

Streak System:
- كل يوم متتالي = +1 streak
- انقطاع يوم = 0 = streak
- Streak freezes: 3 مجانية، تحمي السلسلة من الانقطاع

Achievements:
أضف هذه الإنجازات في seed:

- first_task: "أول مهمة" - أكمل أول مهمة - 10 XP
- first_habit: "أول عادة" - سجل أول عادة - 10 XP
- week_streak: "أسبوع كامل" - 7 أيام متتالية - 50 XP
- month_streak: "شهر كامل" - 30 يوم متتالي - 200 XP
- level_5: "بداية الطريق" - الوصول للمستوى 5 - 50 XP
- level_10: "طالب مجتهد" - الوصول للمستوى 10 - 100 XP
- first_goal: "أول هدف" - أكمل أول هدف - 50 XP
- savings_100: "مدخر" - ادخر $100 - 25 XP
- savings_1000: "مستثمر" - ادخر $1000 - 100 XP

🔄 الـ API Routes

GET /api/user
{ ... ,Response: { id, name, level, xp, title, streaks

PUT /api/user
{ Body: { tawjihiDate, chinaDate, name
Response: updated user

GET /api/habits
Response: habits[] with completions

POST /api/habits
{ Body: { name, icon, color, frequency

Response: created habit

POST /api/habits/[id]/complete ###
{ Response: { success, streak, xpEarned

GET /api/tasks ###
Query: ?status=pending&priority=high
[]Response: tasks

POST /api/tasks ###
{ Body: { title, description, priority, dueDate, category
Response: created task

POST /api/tasks/[id]/complete ###
{ Response: { success, xpEarned, leveledUp

GET /api/transactions ###
Query: ?type=income&category=food
[]Response: transactions

POST /api/transactions ###
{ Body: { type, amount, category, description
Response: created transaction

GET /api/mood ###
[]Response: moodLogs

POST /api/mood ###
{ Body: { mood, energy, stress, note
Response: created moodLog

GET /api/goals ###
Response: goals[] with milestones

POST /api/goals ###
{ Body: { title, description, category, targetDate
Response: created goal

PUT /api/goals/[id]/progress ###
{ Body: { progress
Response: updated goal

POST /api/agent ###
{ []Body: { messages
Response: SSE stream

GET /api/stats ###
{ ... ,Response: { balance, tasksCompleted, habitsLogged, moodAverage

---

✅ قائمة المهام للتنفيذ

1. إنشاء المشروع: npx create-next-app@latest masari --typescript --tailwind --app --src-dir
2. تثبيت Dependencies: npm install prisma @prisma/client zustand framer-motion recharts lucide-react
3. إعداد Prisma: npx prisma init
4. إنشاء Schema
5. تشغيل Migration: npx prisma migrate dev --name init
6. إنشاء Seed للبيانات الأولية
7. إعداد globals.css مع التصميم
8. إنشاء layout.tsx مع RTL
9. إنشاء lib/prisma.ts
10. إنشاء lib/ai-tools.ts
11. إنشاء جميع API routes
12. إنشاء components/layout/sidebar.tsx
13. إنشاء components/layout/header.tsx
14. إنشاء Dashboard
15. إنشاء Agent page
16. إنشاء Habits page
17. إنشاء Tasks page
18. إنشاء Finance page
19. إنشاء Mental page
20. إنشاء RPG page
21. إنشاء Goals page
22. إنشاء China page
23. إنشاء Stats page
24. إنشاء Settings page
25. إضافة Framer Motion animations
26. اختبار جميع الوظائف
27. تحسين Responsive design

⚠️ قواعد مهمة جداً

- RTL إجباري: كل النصوص من اليمين لليسار
- Database فقط: لا تستخدم mock data أبداً
- AI Agent حقيقي: يستخدم z-ai-web-dev-sdk فعلياً
- XP system يعمل: حساب صحيح وترقية تلقائية
- Streak system يعمل: تتبع الأيام المتتالية
- Design clean: لا زخارف، تصميم نظيف مثل Linear

- Responsive: يعمل على جميع الشاشات
- Arabic: كل النصوص بالعربية

ابدأ الآن! أنشئ المشروع من الصفر واتبع القائمة.
ركز على أن كل شيء يعمل فعلياً قبل أن يبدو جميلاً.
.Less is More. Functionality First

استغمل هذا ال: Grq api key
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